Multimedia Systems Homework

(Due at 23:55 on November 1)

Total Score (20 points)

What you may and may not use

Parts 1–4 are one program, written individually. That program turns a mathematical description of shapes into an array of pixels. You must write that conversion yourself.

You may use a library to hold the pixel array (numpy — or just use a plain nested list), to save that array to an image file, to plot the measurements you are asked for, and to build a user interface.

You may not call anything that decides for you **which pixels a shape covers**: *line(), polygon(), ellipse(), bezier(), fill()*, path stroking, or any built-in anti-aliasing or smoothing flag. Writing values into your own array — including a whole row or a whole slice at a time — is exactly what you are supposed to do. Part 1 is the exception, but only for drawing: there you may hand a plotting library the line segments and sampled points you have already computed, because no pixel coverage is being graded yet. The curve coordinates themselves must still come from your own two implementations — bezier() and any other built-in curve routine stay banned throughout.

Rules

1. Comment your code clearly. Your comments and write-up may be in Chinese or English.
2. Using a drawing function that the "What you may and may not use" section rules out means 0 points for every item it was used for.
3. You may use AI tools to write your code. You are responsible for **every line** you submit — "the AI wrote it" is not an acceptable defense, and you may be asked to explain any part of your program in person.
4. Every number and every figure this assignment asks you to report must come from running **your own program**. A number your submitted code and figures do not support receives no credit for that item.
5. You may **not** submit another student's work. If two submissions are highly similar, both students will be asked to do so under Rule 3; those who cannot will receive 0 points.

Submission Format

1. For **Parts 1–4**, submit your source code, an executable file, or a one-line command that runs the source, and **one PDF file** containing: how to build it, with the OS, software and environment variables involved; every figure and every number the parts ask for; your answers to the questions in Parts 1–3; and your Part 4 write-up.
2. Attach a separate PDF file for Part 5 (team member list, project description). Each member may submit the same file, inside their own folder.
3. Create a folder that contains your files, name the folder with your Student ID and name (for example: M11509001_yourname), and then compress it into a ZIP. Upload it to Moodle before the deadline.

Part 1: Two Roads to the Same Curve

Total Score (4 points)

Chapter 4 gives two constructions for the same Bézier curve, and never says that they are equivalent.

A. The cubic formula (Figure 1, left): $C(t) = (1-t)^3 b_0 + 3t(1-t)^2 b_1 + 3t^2(1-t)b_2 + t^3 b_3$

B. The mid-point construction (Figure 1, right): take mid-points of the control polygon to split the curve at t = 0.5 into two Bézier curves with control points of their own, then recurse to a given depth. The polyline through the endpoints of the resulting sub-curves is your approximation.

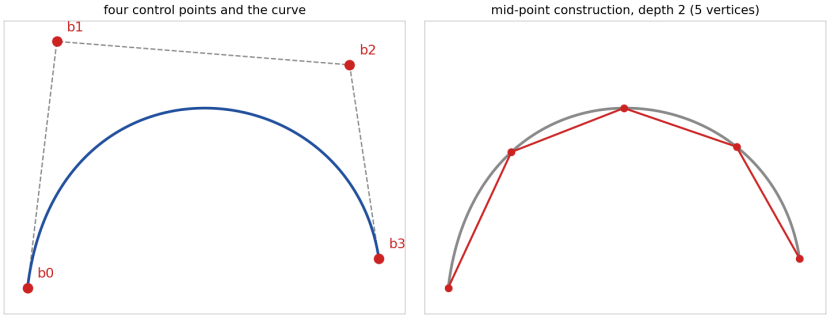


Figure 1: (left) four control points and the curve they define;
(right) the mid-point construction stopped early, drawn over the true curve

1. Implement both. Choose four control points that give a clearly curved shape spanning most of your canvas, and show the curve each construction produces (one overlaid figure is enough, at a shallow depth where the two are still visibly different). (1 point)
2. Convergence. Overlay B on A. Measure the gap this way: **for every point on the formula curve, take its distance to the nearest segment of your polyline; report the largest** — in that direction, not the reverse. Sample A finely enough — consecutive points under 0.1 px apart — or you are measuring your own sampling of A, not the gap. Report it as a function of subdivision depth, then raise the depth until it falls below **0.5 px** and report that depth. State the rule your errors follow each time the depth increases by one, and whether the shallow depths obey it as closely as the deep ones. Finally: the formula makes you decide in advance how many values of t to sample. Name one thing the mid-point construction lets you do that this does not. (1.5 points)
3. Joins. Build an open path of **three** Bézier segments; that gives you two joins. Make one smooth and one a corner point, and say in one line what makes the corner a corner, in terms of its direction lines. Chapter 4 states the condition for a smooth join as “control points are lined up *and* direction lines are of the same length”. Isolate one half of that: keep the join point and the two direction points on either side of it collinear, but make the two direction lines clearly unequal in length. Show it next to the equal-length version and say whether the unequal-length join still looks smooth. If it does, explain what the equal-length half buys you, given that the shape does not need it. (1.5 points)

Part 2: Inside and Outside

Total Score (5 points)

1. Implement a scanline fill for a closed path using the **non-zero winding number rule**, as Chapter 4 defines it (its "line" must be read as a ray — a full line would count the crossings on both sides): draw a ray from the point you are testing in any direction; add one each time the path crosses it from left to right, subtract one each time it crosses from right to left; non-zero means inside. *Scanline* means you work one row of pixels at a time instead of one pixel at a time: find where the path crosses the row, then walk along the row accumulating the signed crossings, and fill wherever the **running total is non-zero**. That running total is the winding number, so "crosses from left to right" becomes a statement about whether an edge passes through the row upwards or downwards — settle once which of the two is +1 in your coordinate system, and keep it. Filling *alternate* spans between crossings is a different rule — that is even-odd — and on a self-intersecting path it gives a different picture; item 2 is such a path. Your fill must work for a path built from more than one sub-path. (3 points)
2. Draw a five-pointed star as a **single self-intersecting closed path** — five vertices visited in "every other one" order, not ten alternating inner and outer vertices (Figure 2, left). Fill it. Show the result and give the winding number at the center — its sign follows from the convention you settled on in item 1, and either sign is acceptable. (1 point)

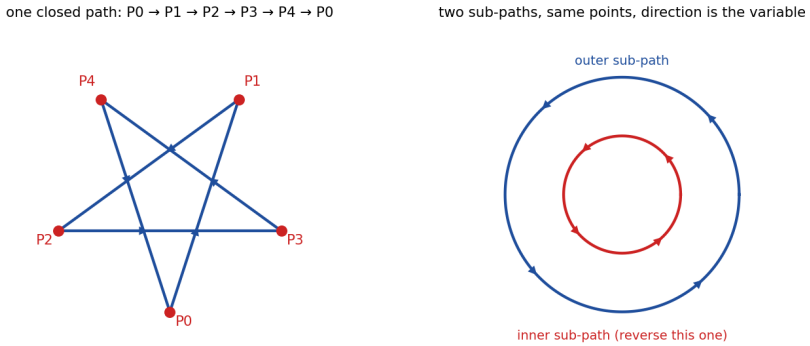


Figure 2: the two paths you need. Arrows show the direction of travel.
In the star, the labels are numbered in visiting order — each step skips one vertex around the circle.
Neither is shown filled — what the fill does to them is what you are asked to find out.

3. Chapter 4 ends the winding rule with one sentence: *"Depends on the path's having a direction."* Make that sentence visible. Build a path from two sub-paths, an outer circle and a smaller inner circle (Figure 2, right). Describe each circle as four Bézier segments — the two direction points sit 0.5523·r along the tangent from each endpoint — and flatten them with your Part 1 construction; that is the whole pipeline, end to end. Render it twice, changing **nothing but the direction** in which the inner circle is traversed — not one point may move. Show both results, give the winding number at the center in each case, and explain the difference. Then say what this implies about how the letter "O" has to be stored in a font file. (1 point)

Part 3: Landing on the Pixel Grid

Total Score (6 points)

Chapter 2 says that sampling below the Nyquist rate causes aliasing, and that in images this is "usually seen in the form of jagged edges". Chapter 4 says of anti-aliasing that "Brightness is proportional to area of intersection". This part is about how those are two halves of one statement, not two separate facts.

I. Area coverage (2 points)

a. Rotate your star from Part 2.2 by a few degrees — 5° is enough — so that no edge is exactly horizontal or vertical, and use that rotated star wherever Part 3 asks for the star (I and II; III uses a different shape). A regular five-pointed star drawn as in Figure 2 has one *exactly horizontal* edge, and an axis-aligned edge is the one case where the staircase never appears at all.

b. Then approximate Chapter 4's rule by supersampling: a pixel's value is the fraction of k × k sub-positions inside it that the shape covers. The easy way to get that is to run your Part 2 fill on a canvas k times larger in each direction, with the shape scaled by k as well, and average each k × k block down to one pixel — the same computation, without testing sub-positions one at a time. Render the rotated star at k = 1, 2, 4 and 8, and show all four. **k = 8 is what II and III below mean by your largest k.**

II. Measure it (1.5 points)

a. Find the shallowest edge of your rotated star and pick a pixel row that crosses it **near one of its ends** — along the middle of that edge the star is filled on both sides, the core on one side and a spike on the other, so there is no transition there to plot. Plot the values along that row — **scan along the edge, not across it**. If other edges of the star cross the same row, plot only the stretch of the row around the edge you picked. Do this for k = 1 and for your largest k, and describe how the two plots differ in shape. (Scanning across the edge instead gives a transition one pixel wide at every k, so the two plots come out nearly the same.)

b. Report the render time for each k, on one canvas size that you state. Say how the cost grows with k, and why.

III. Where the grid loses (2.5 points)

Render a shape whose detail becomes finer than one pixel: a wheel of at least 40 wedges radiating from a center point, **separated by gaps as wide as the wedges** (Figure 3). Each wedge can be a triangle — the center and two points on a circle — so the wheel is just a path with 40-odd sub-paths. Render it at k = 1 and at your largest k, and attach both. If the center of your wheel does not visibly break down at k = 1, add more wedges until it does.

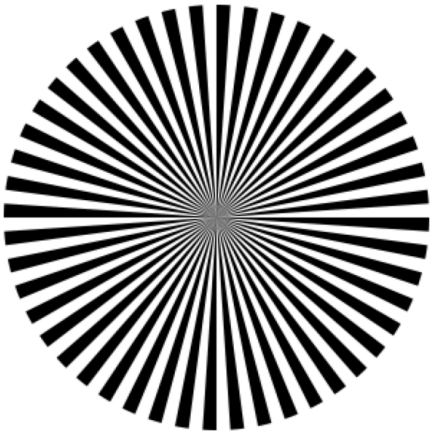


Figure 3: a wheel of 48 wedges, rendered with heavy supersampling

a. At k = 1 you will see something that is not merely jagged edges. Name it.

b. Raising k cleans up the outside of the wheel, but however large k gets it never *recovers* the wedges at the center: what you named in (a) gives way to something closer to a uniform gray, never to the picture itself. Explain why, using Chapter 2: say how the spatial frequency behaves as you move toward the center, and what the Sampling Theorem therefore says about this picture.

c. Chapter 2 also lists filters. Which kind is relevant here, and where in the process does the area-coverage rule you implemented in I already apply it? Given your answer to (b), say what it does and does not achieve.

Part 4: One Thing You Wanted to Know

Total Score (2 points)

Add one thing to your renderer and report what you found out. Pick from the list or propose your own — say which at the top of your write-up. Half a page is enough.

- **Stroke.** (Chapter 4) Implement line caps (butt / round / projecting) and joins (miter / round / bevel). Then make the angle at a miter join very sharp, and report what happens to the join.
- **Transformations.** (Chapter 4) Implement rotation about an arbitrary point. Rotate your star's vertices by 1° a hundred times and rasterize once at the end. Then do it the other way round: rasterize first, then rotate the resulting bitmap by 1° a hundred times, resampling every time (write that resampling yourself). Compare the two results and put a number on the difference. Chapter 3 predicts which one survives — say whether your measurement agrees.
- **Layers and transparency.** (Chapter 3) Composite two shapes with partial transparency. Show that swapping their order changes the result, and say exactly which pixels change.
- **Interactivity.** (Chapter 1) Make the Bézier control points draggable with live redraw. Report one thing about the control points that you could see this way and could not see in a static picture.
- **Vectorizing.** (Chapter 3) The chapter says "for pixel to vector is more problematical". Try to recover the outline of your star from your own bitmap by following edges. Report how well it worked and what specifically defeated it.

Graded on whether you actually ran it and whether you report something you did not know before — **not on how ambitious it is.**

Part 5: Final Project & Team Member List

Total Score (3 points)

1. Team member list should include the following information: (1 point)
   - Student ID
   - Class (Univ., College/School, and Department)
   - Name
   - Email
   - Topic of the final project
2. Briefly describe your final project, including its purpose, input, and output (less than 500 words). (2 points)